

LAPORAN PRAKTIKUM SISTEM OPERASI

MODUL 9 SYSCALL XINU



Disusun Oleh :

Ilham Lii Assidaq 2311104068

Dosen Pengampu :

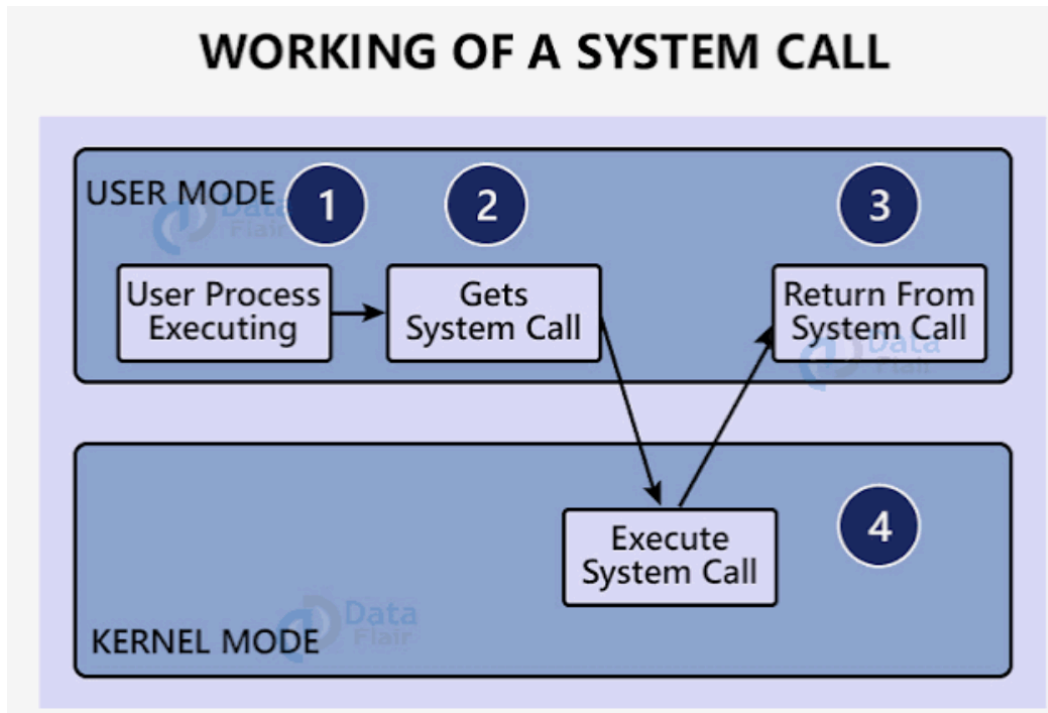
Yudha Islami Sulistya, S.Kom., M.Cs

PROGRAM STUDI S1 SOFTWARE ENGINEERING

FAKULTAS INFORMATIKA

TELKOM UNIVERSITY PURWOKERTO

1. Dasar Teori



System Call (Syscall) adalah antarmuka layanan yang disediakan oleh sistem operasi sebagai jembatan pembatas antara user space (ruang aplikasi) dan kernel space (ruang inti OS). Syscall berfungsi layaknya gerbang pengaman yang mencegah aplikasi pengguna mengakses memori atau perangkat keras secara langsung, sehingga stabilitas dan keamanan sistem tetap terjaga tanpa aplikasi perlu mengetahui detail rumit di dalam kernel.

Agar operasi kernel tetap konsisten, Xinu menjalankan Syscall melalui mekanisme proteksi yang ketat. Alur kerjanya dimulai dengan mematikan semua interupsi (interrupt) guna melindungi struktur data global, dilanjutkan dengan memvalidasi argumen proses untuk mencegah manipulasi. Setelah itu, kernel mengeksekusi layanan inti (seperti memodifikasi prioritas atau memori), mengaktifkan kembali interupsi agar sistem kembali normal, dan terakhir mengembalikan status keberhasilan atau pesan error (SYSERR) kepada proses pemanggil.

Fungsi Utama Syscall

- Akses Layanan: Menjadi perantara bagi program untuk meminta alokasi memori, mengelola proses, atau melakukan operasi Input/Output (I/O).
- Perlindungan Keamanan: Membatasi program agar tidak merusak sistem atau mengakses memori milik aplikasi lain.

- Information Hiding: Menyembunyikan detail internal operasi kernel sehingga pengguna hanya perlu menerima hasil akhirnya saja.

5 Langkah Cara Kerja Syscall di Xinu

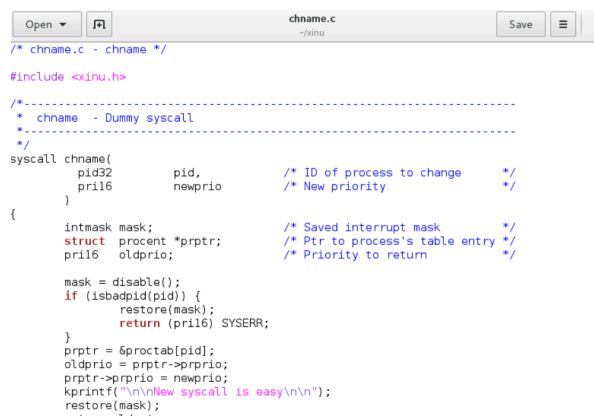
1. Mematikan Semua Interrupt: Mencegah gangguan dari proses lain saat kernel mengubah data penting agar OS tidak mati (crash).
2. Memeriksa Argumen: Melakukan validasi parameter input agar sistem operasi tidak mudah dimanipulasi.
3. Menjalankan Layanan Utama: Mengeksekusi instruksi inti seperti mengubah status proses (resume()), melepas memori (freemem()), atau mengganti prioritas (chprio()).
4. Mengaktifkan Interrupt Kembali: Membuka kembali akses penjadwalan (scheduler) agar proses-proses lain bisa berjalan bergantian.
5. Memberikan Hasil ke Proses: Mengirimkan laporan balik ke aplikasi berupa nilai sukses atau tanda kegagalan sistem (SYSERR).

2. Manfaat

Melalui praktikum pada modul ini, manfaat yang saya dapatkan adalah mampu memahami peran krusial System Call sebagai jembatan pengaman antara user space dan kernel space, sehingga saya mengetahui bagaimana sistem operasi Xinu melindungi integritas data global dan memvalidasi setiap permintaan layanan dari aplikasi pengguna secara terstruktur. Selain itu, pemahaman mendalam mengenai lima langkah alur kerja syscall atomik ini secara nyata meningkatkan keterampilan analisis dan logika saya dalam merancang program berbasis jaringan atau manajemen proses yang aman, stabil, serta terhindar dari risiko kerusakan sistem.

3. Jurnal

- A. [50 Poin] Buat syscall baru seperti yang ditunjukkan pada modul syscall poin 9.5! (sertakan Screenshot kode dan hasil run)



```

chname.c
~Xinu

/* chname.c - chname */

#include <xinu.h>

/*-----
 * chname - Dummy syscall
 *-----*/
syscall chname(
    pid32      pid,      /* ID of process to change */
    pri16      newprio   /* New priority */
)
{
    intmask mask;        /* Saved interrupt mask */
    struct proctab *prptr; /* Ptr to process's table entry */
    pri16      oldprio;   /* Priority to return */

    mask = disable();
    if (isbadpid(pid)) {
        restore(mask);
        return (pri16) SYSERR;
    }
    prptr = &proctab[pid];
    oldprio = prptr->prprio;
    prptr->prprio = newprio;
    kprintf("\n\nNew syscall is easy\n\n");
    restore(mask);
    return oldprio;
}

```

B. [25 Poin] Perbaiki syscall chprio (xinu/system/chprio.c) dengan memperhatikan validasi input

- Pastikan id adalah angka dari 0 – NPROC (ukuran maks banyaknya proses)
- Pastikan prioritas adalah bilangan yang positif

C. Lakukan hal-hal berikut ini

- Edit xsh_uptime.c

Tambahkan kode berikut

```
/* Check for valid number of arguments */  
  
if (nargs > 1) {  
    fprintf(stderr, "%s: too many arguments\n", args[0]);  
    fprintf(stderr, "Try '%s --help' for more information\n",  
            args[0]);  
    return 1;  
}  
  
// tambahan code  
chprio(5,33) // memanggil syscall chprio; mengubah prioritas menjadi 33 pada id=5
```

- Compile source code tersebut dengan perintah

make clean

make

- Jalankan perintah ps

xsh \$ ps

perhatikan prioritas proses dengan id = 5

- Jalankan uptime

xsh \$ uptime

- Perhatikan hasil perintah tersebut

Jalankan ps

xsh \$ ps

- perhatikan prioritas proses dengan id = 5 seharusnya sudah berubah

[25 Poin] Testing chprio syscall yang telah diubah

- Testing prioritas tidak boleh < 0: Ubah “chprio(5,33)” menjadi “chprio(5,-3)” pada xsh_uptime.c

- Testing id adalah valid: Ubah “chprio(5,33)” menjadi “chprio(3000,3)”

- Hasil dua testing di atas adalah prioritas tidak berubah karena salah argument (dibuktikan dengan menggunakan perintah ps)

```
xinu@xinu-develop-end: ~/xinu/system
File Edit View Search Terminal Help
xinu@xinu-develop-end:~/xinu/compile$ cd ..
xinu@xinu-develop-end:~/xinu$ cd system
xinu@xinu-develop-end:~/xinu/system$ ls
ascdat.c      DESCRIPTION  gettimeofday.c  mkbufpool.c    read.c          sleep.c
bufinit.c     device      1386.c         net             ready.c         stacktrace.c
chname.c      evect.c     include        newqueue.c     receive.c       start.S
chprio.c      exit.c      init.c         open.c         recvclr.c       suspend.c
clkdisp.S     freebuf.c   initialize.c   panic.c        recvtime.c      system
clkhandler.c  freemem.c   insert.c       pci.c          resched.c       unsleep.c
clkinit.c     getbuf.c    insertd.c      ptclean.c      resume.c        userret.c
close.c       getc.c      intr.S         ptcoun.c       seek.c          wait.c
compile       getdev.c    ioerr.c        ptcreate.c     semcount.c      wakeup.c
conf.c        getitem.c   ionull.c       ptdelete.c     semcreate.c     write.c
config        getmem.c    kill.c         ptinit.c       semdelete.c     xdone.c
control.c     getpid.c    kprintf.c      ptrece.c       semreset.c      yield.c
COPYRIGHT     getprio.c   lib            ptnet.c        send.c
create.c      getstk.c    main.c         ptnet.c        shell
ctxsw.S       getticks.c  mark.c         ptnet.c        signal.c
debug.c       gettime.c   meminit.c      queue.c        signaln.c
xinu@xinu-develop-end:~/xinu/system$ gedit chprio.c
```

```
*chprio.c
~/xinu/system
Save

pid32 pid, /* id of process to change */
pri16 newprio /* New priority */

}

intmask mask; /* Saved interrupt mask */
struct procent *prptr; /* Ptr to process's table entry */
pri16 oldprio; /* Priority to return */

mask = disable();

/* Validasi: id harus antara 0 sampai NPROC */
if (isbadpid(pid) || pid < 0 || pid >= NPROC) {
    restore(mask);
    return (pri16) SYSEERR;
}

/* Validasi : Prioritas harus positif (> 0) */
if (newprio <= 0) {
    restore(mask);
    return (pri16) SYSEERR;
}

prptr = &proctab[pid];
oldprio = prptr->prprio;
prptr->prprio = newprio;
restore(mask);
return 0xffff & oldprio;
}
```

```
xinu@xinu-develop-end: ~/xinu/compile
File Edit View Search Terminal Help
-00 -DBRELOC=0x150000 -DBOOTPLOC=0x150000 -DBSDURG -DVERSION=""cat version"\ -I../i
nclude -o binaries/xsh udpecho.o ../shell/xsh udpecho.c
/usr/bin/gcc-4.8 -march=i586 -m32 -fno-builtin -fno-stack-protector -nostdlib -c -Wall
-00 -DBRELOC=0x150000 -DBOOTPLOC=0x150000 -DBSDURG -DVERSION=""cat version"\ -I../i
nclude -o binaries/xsh udpserver.o ../shell/xsh udpserver.c
/usr/bin/gcc-4.8 -march=i586 -m32 -fno-builtin -fno-stack-protector -nostdlib -c -Wall
-00 -DBRELOC=0x150000 -DBOOTPLOC=0x150000 -DBSDURG -DVERSION=""cat version"\ -I../i
nclude -o binaries/xsh uptime.o ../shell/xsh uptime.c
/usr/bin/gcc-4.8 -march=i586 -m32 -fno-builtin -fno-stack-protector -nostdlib -c -Wall
-00 -DBRELOC=0x150000 -DBOOTPLOC=0x150000 -DBSDURG -DVERSION=""cat version"\ -I../i
nclude -o binaries/clkdisp.o ../system/clkdisp.S
/usr/bin/gcc-4.8 -march=i586 -m32 -fno-builtin -fno-stack-protector -nostdlib -c -Wall
-00 -DBRELOC=0x150000 -DBOOTPLOC=0x150000 -DBSDURG -DVERSION=""cat version"\ -I../i
nclude -o binaries/ctxsw.o ../system/ctxsw.S
/usr/bin/gcc-4.8 -march=i586 -m32 -fno-builtin -fno-stack-protector -nostdlib -c -Wall
-00 -DBRELOC=0x150000 -DBOOTPLOC=0x150000 -DBSDURG -DVERSION=""cat version"\ -I../i
nclude -o binaries/intr.o ../system/intr.S
/usr/bin/gcc-4.8 -march=i586 -m32 -fno-builtin -fno-stack-protector -nostdlib -c -Wall
-00 -DBRELOC=0x150000 -DBOOTPLOC=0x150000 -DBSDURG -DVERSION=""cat version"\ -I../i
nclude -o binaries/ttydispatch.o ../device/tty/ttydispatch.S
/usr/bin/gcc-4.8 -march=i586 -m32 -fno-builtin -fno-stack-protector -nostdlib -c -Wall
-00 -DBRELOC=0x150000 -DBOOTPLOC=0x150000 -DBSDURG -DVERSION=""cat version"\ -I../i
nclude -o binaries/ethdispatch.o ../device/eth/ethdispatch.S

Loading object files to produce GRUB bootable xinu
Copying xinu.elf to xinu.boot in the TFTP directory.
xinu@xinu-develop-end:~/xinu/compile$
```

```
xinu@xinu-develop-end: ~/xinu/shell
File Edit View Search Terminal Help
xinu@xinu-develop-end:~/xinu$ cd shell
xinu@xinu-develop-end:~/xinu/shell$ ls
addargs.c      xsh_cat.c      xsh_help.c      xsh_ns.c      xsh_udpdump.c
lexan.c        xsh_clear.c    xsh_kill.c      xsh_ping.c    xsh_udpecho.c
Notes_On_Pipes xsh_date.c     xsh_ls.c        xsh_ps.c      xsh_udpserver.c
shell.c        xsh_devdump.c  xsh_memdump.c   xsh_rdstest.c xsh_uptime.c
xsh_argecho.c  xsh_echo.c     xsh_memstat.c   xsh_sleep.c
xsh_arp.c      xsh_exit.c     xsh_netinfo.c   xsh_tee.c
xinu@xinu-develop-end:~/xinu/shell$ gedit xsh_uptime.c
```

```
*xsh_uptime.c
~/xinu/shell

printf("Use: %s\n\n", args[0]);
printf("Description:\n");
printf("\tDisplays time since the system booted\n");
printf("Options:\n");
printf("\t--help\t display this help and exit\n");
return 0;
}

/* Check for valid number of arguments */
if (nargs > 1) {
    fprintf(stderr, "%s: too many arguments\n", args[0]);
    fprintf(stderr, "Try '%s --help' for more information\n",
            args[0]);
    return 1;
}

/* Tambahkan code disini*/
chprio(5,33); |

secs = clktime;          /* Total seconds since boot */

/* Subtract number of whole days */

days = secs/secperday;
secs -= days*secperday;

C Tab Width: 8 Ln 40, Col 23 INS
```

```
Welcome to Xinu!

xsh $ ps
Pid Name          State Prio Ppid Stack Base Stack Ptr  Stack Size
-----
0 prnull          ready  0    0  0x005FDFFC 0x00FFFE4    8192
1 ipout           wait   500   0  0x005FBFFC 0x005FBF30   8192
2 netin           wait   500   0  0x005F9FFC 0x005F9ED0   8192
4 Main process    recv   20    3  0x005E7FFC 0x005E7F44  65536
5 shell           recv   50    4  0x005D7FFC 0x005D79AC   8192
6 ps              curr   20    5  0x005F7FFC 0x005F7FC4   8192
xsh $

CTRL-A 7 for help | 115200 8N1 | NOR | Minicom 2.7 | VT102 | Online 0:0 | ttvS0
```

```
File Edit View Search Terminal Help

xsh $ ps
Pid Name          State Prio Ppid Stack Base Stack Ptr Stack Size
-----
0 prnull          ready  0    0 0x005FDFFC 0x00FFFE4 8192
1 ipout           wait   500  0 0x005FBFFC 0x005FBF30 8192
2 netin           wait   500  0 0x005F9FFC 0x005F9ED0 8192
4 Main process    recv   20   3 0x005E7FFC 0x005E7F44 65536
5 shell           recv   50   4 0x005D7FFC 0x005D79AC 8192
6 ps              curr   20   5 0x005F7FFC 0x005F7FC4 8192

xsh $ uptime
Xinu has been up 15 second(s)

New Syscall is easy

xsh $ ps
Pid Name          State Prio Ppid Stack Base Stack Ptr Stack Size
-----
0 prnull          ready  0    0 0x005FDFFC 0x00FFFE08 8192
1 ipout           wait   20   0 0x005FBFFC 0x005FBF30 8192
2 netin           wait   500  0 0x005F9FFC 0x005F9ED0 8192
4 Main process    recv   20   3 0x005E7FFC 0x005E7F44 65536
5 shell           recv   33   4 0x005D7FFC 0x005D79AC 8192
8 ps              curr   20   5 0x005F7FFC 0x005F7FC4 8192

xsh $
```

```
xsh $ ps
Pid Name          State Prio Ppid Stack Base Stack Ptr Stack Size
-----
0 prnull          ready  0    0 0x005FDFFC 0x00FFFE08 8192
1 ipout           wait   500  0 0x005FBFFC 0x005FBF30 8192
2 netin           wait   500  0 0x005F9FFC 0x005F9ED0 8192
4 Main process    recv   20   3 0x005E7FFC 0x005E7F44 65536
5 shell           recv   50   4 0x005D7FFC 0x005D79AC 8192
6 ps              curr   20   5 0x005F7FFC 0x005F7FC4 8192

xsh $ uptime
Xinu has been up 9 second(s)

New Syscall is easy

xsh $ ps
Pid Name          State Prio Ppid Stack Base Stack Ptr Stack Size
-----
0 prnull          ready  0    0 0x005FDFFC 0x00FFFE08 8192
1 ipout           wait   20   0 0x005FBFFC 0x005FBF30 8192
2 netin           wait   500  0 0x005F9FFC 0x005F9ED0 8192
4 Main process    recv   20   3 0x005E7FFC 0x005E7F44 65536
5 shell           recv   50   4 0x005D7FFC 0x005D79AC 8192
8 ps              curr   20   5 0x005F7FFC 0x005F7FC4 8192

xsh $
```

4. Kesimpulan

Berdasarkan praktikum yang telah saya laksanakan pada modul ini, saya dapat menyimpulkan bahwa saya telah berhasil memahami peran penting System Call sebagai antarmuka yang menjembatani interaksi aman antara user space dan kernel space pada sistem operasi Xinu. Melalui analisis langsung terhadap dokumen, saya kini memahami bagaimana kernel mengimplementasikan lima langkah alur kerja yang ketat untuk menjaga konsistensi data global sistem. Proses krusial seperti mematikan interupsi secara atomik, melakukan validasi argumen untuk mencegah manipulasi, hingga mengeksekusi fungsi inti seperti resume atau chprio telah memberikan saya wawasan mendalam mengenai mekanisme perlindungan

keamanan dan penyembunyian informasi internal kernel. Pengalaman teknis ini secara nyata meningkatkan kemampuan analisis saya dalam memahami ritme kerja sistem operasi tingkat rendah saat menanggapi permintaan layanan dari aplikasi pengguna secara aman dan terstruktur.